Detection of Inconsistencies in API Logs Using Machine Learning

Ayush Bharti
Chitkara University
ayush204.be22@chitkara.edu.in

Sonalika Singh
Chitkara University
sonalika2398.be22@chitkara.edu.in

Abstract

APIs are central to how modern software systems communicate, and every API call leaves behind a log entry that records what happened. In large-scale systems, these logs pile up fast — we are talking millions of entries per day — which makes manual inspection completely impractical. Missing inconsistencies in these logs can mean undetected security issues, bugs that go unreported, or performance problems that nobody knows about until it is too late. This paper proposes a two-layer detection system that pairs a rule-based checker with the Isolation Forest machine learning algorithm to automatically find inconsistencies in API log data. The raw logs go through a cleaning and feature engineering pipeline before the model runs. The rule layer catches clear workflow violations, and Isolation Forest picks up statistical anomalies the rules miss. Testing on a real-world dataset of roughly one million web server log entries gave 93% accuracy, 95% precision, 90% recall, and a 92% F1 -score on a held-out test set. The results suggest this kind of hybrid setup works well in practice and could be deployed as part of a real monitoring pipeline.

Keywords: API logs, anomaly detection, Isolation Forest, machine learning, system monitoring, feature engineering, session analysis

1. Introduction

1.1 Background

Over the past decade or so, the way software gets built has changed quite a lot. Most applications used to be one big codebase — everything packed together and deployed as a single unit. That worked okay for smaller systems but became a real headache as things scaled up. Now the standard approach is to break an application into smaller, independent services that each handle one specific job, and have them talk to each other using APIs. This design pattern, commonly called microservice architecture, is how companies like Amazon, Netflix and Uber run their backends. The side effect is that APIs have become the connective tissue of pretty much every modern software system.

Every API call produces a log entry. These entries usually contain things like the source IP address, a timestamp, the HTTP method used, which endpoint was called, the status code the server returned, how many bytes were transferred, and some information about the client making the request. Taken together, all of these logs give engineers a fairly complete picture of what is happening in the system at any given time. They are genuinely useful for debugging, performance analysis, and security auditing — but only if someone actually looks at them, which brings us to the core problem.

Anomaly detection, at its core, is about finding things that do not look right compared to what normally happens. In the context of API logs, the kinds of anomalies we care about are things like a client accessing a protected resource without logging in first, API calls arriving in an order that does not match the expected workflow, or traffic volumes from a single session that are wildly higher than normal. These are what we call inconsistencies throughout this paper. If they go unnoticed, they can signal anything from a security breach to a misbehaving third-party integration to a plain old developer mistake.

The volume issue makes everything harder. A reasonably busy e-commerce site can produce hundreds of thousands of log entries per hour. Large platforms and cloud API gateways are dealing with millions per day. There is no realistic way to have a person review all of that. The automated monitoring tools most teams use today are mostly threshold-based — if error rate crosses X% or response time exceeds Y milliseconds, send an alert. That is useful for catching obvious failures, but it completely misses a whole class of problems that do not register as metric spikes at all.

That missing category is what we focus on here. Take a few concrete examples. A session might be hitting /user/orders repeatedly without ever going through /login first — a pretty obvious authentication bypass attempt. Or a client might jump straight to /checkout without any prior /cart/add activity, which is not how any real shopping session works. Or one IP address could be firing off 400 API calls in under a minute, which strongly suggests scraping or bot traffic. None of these scenarios would necessarily trigger a response-time alert or an error rate spike. But they are all real signs of something going wrong, whether that is a security problem, a broken client, or a developer who did not test their integration properly. Building a system to catch this automatically is what motivated this work.

1.2 Literature Survey

There is quite a bit of existing work on anomaly detection in logs and network traffic, so it is worth summarising what has been tried and where things currently stand. Nassif et al. (2021) [6] did a large systematic review of ML-based anomaly detection methods and found that no single algorithm consistently wins across different datasets and problem types. Their key takeaway was that hybrid approaches — ones that combine multiple strategies — tended to give more stable results. That finding is basically the starting point for the approach taken in this paper.

On the log-specific side, DeepLog (Du et al., 2017) was one of the first well-known attempts to use LSTM networks for log anomaly detection. The idea was to train a model on sequences of normal log events and flag anything that looked out of order. It worked reasonably well and showed that the ordering of events in a log carries real information. LogAnomaly (Meng et al., 2019) pushed this further by looking at both the order of events and their numerical properties at the same time, which improved detection rates on several benchmark datasets.

One thing that often gets overlooked in log anomaly detection papers is how the raw logs get turned into something a model can actually use. He et al. (2017) [21] tackled this with Drain, a parsing tool that converts unstructured log text into structured templates using a fixed-depth parse tree. It is fast and reasonably accurate, and it has become something of a standard pre-processing step in the log analysis

community. The ingestion pipeline used in this paper follows a similar approach, using regex-based parsing to extract structured fields from raw Nginx log lines.

A recurring problem with sequence-based approaches is that they tend to break when the log format changes, which happens a lot in practice as developers update their code. LogRobust (Zhang et al., 2019) dealt with this by representing log events as semantic vectors rather than raw tokens, making the model more resilient to format shifts. More recently, transformer-based methods have taken this even further — LogBERT (Guo et al., 2021) applies masked language modelling borrowed from NLP to learn log patterns without needing any labeled anomaly data, and NeuralLog (Le and Zhang, 2021) goes one step further by skipping the template extraction step entirely.

For HTTP traffic specifically, Kim and Cho (2018) [5] built a C-LSTM model that stacks convolutional and recurrent layers to pick up both spatial and sequential patterns in web traffic. The results were good but the approach needs a lot of labeled training data, which is usually not available in real deployments. Tama et al. (2020) [4] went a different route with a stacked ensemble of classifiers, showing that combining diverse models helped generalise better than any individual one. Both approaches are supervised though, which means they depend on having labeled examples of anomalies — something that is hard to come by in practice. Using Isolation Forest, which is unsupervised, sidesteps that issue.

Work targeting distributed microservice environments has gone in a few interesting directions. DeepTraLog (2022) combined regular log data with distributed tracing information and used graph-based learning to detect anomalies that span multiple services at once — something that single-service approaches would miss entirely. Carrera et al. (2022) [7] showed that you can get solid detection results by combining multiple unsupervised methods in a pipeline, even when you have no labeled ground truth to train on. That is an important practical result because in most production systems you never have a clean set of labeled anomaly examples.

More recently, API-TAD (2023) has focused specifically on endpoint-level patterns in API traffic rather than generic log streams, which is a step closer to the problem addressed in this paper. Work in regulated domains like open banking has also shown that ensemble classifiers such as Random Forest can work well for detecting unauthorized API access, though these approaches require labeled training data. Inuwa and Das (2024) [8] compared several ML methods for detecting cyber anomalies in IoT traffic and again found that no single method dominates — the right choice depends on the properties of the data you are working with. On the explainability side, approaches like MINES (2025) have started trying to make anomaly detection outputs more interpretable so that analysts can understand why something was flagged, not just that it was.

Looking specifically at the algorithm used in this study, Lu et al. (2023) [2] reviewed several anomaly detection methods for streaming data and noted that isolation-based methods tend to perform well when anomalies are sparse and the data is high-dimensional — which matches the API log scenario pretty well. Trivedi and Shah (2024) [1] looked at both deep learning and classical ML approaches for web traffic tasks and pointed out that while deep models often get better numbers, they come at the cost of higher compute and worse interpretability, which makes simpler hybrid approaches more practical for day-to-day monitoring. Chabchoub et al. (2022) [18] went back to the original Isolation Forest algorithm and

proposed some improvements to how it partitions the feature space, providing additional theoretical backing for why this algorithm is a reasonable choice for this kind of task.

Looking across all of this work, one gap stands out. Most of the existing systems are built for generic system logs or raw network traffic. Very few of them deal specifically with the logical structure of API call sequences — things like whether authentication happened before resource access, or whether checkout was preceded by cart activity. And almost all of them use either a rule-based approach or a statistical model, but not both in combination. The idea of pairing a rule checker with a statistical anomaly detector in the same pipeline is what this paper explores.

1.3 Research Gap

Based on the literature review, four specific gaps stand out. First, most existing systems treat logs as generic event streams and ignore API-specific structure like endpoint semantics, HTTP method constraints, and expected request sequences. Second, the focus in the literature is mostly on statistical anomalies — traffic spikes and response time outliers — while logical inconsistencies in API workflows get much less attention. Third, the methods that do work well tend to be supervised, which means they need labeled anomaly data that most real production teams simply do not have. Fourth, and most relevant to this paper, there are very few examples of systems that combine explicit rule checking with data-driven anomaly detection in a single pipeline. That last gap is what this work tries to address.

1.4 Objectives

Based on these gaps, this study has five main goals:

1. To identify and engineer features from API log data that capture both structural characteristics and behavioral usage patterns.
2. To design a rule-based validation layer that enforces known API workflow constraints, including authentication sequencing, HTTP method correctness, and endpoint access ordering.
3. To apply the Isolation Forest algorithm as a complementary statistical anomaly detection layer operating on the engineered feature space.
4. To integrate the two layers into a unified hybrid framework and evaluate whether their combination outperforms each component independently.
5. To assess the effectiveness of the proposed system on a large-scale real-world web server log dataset using standard classification metrics.

1.5 Scope

This study focuses on inconsistencies that can be detected using structured log data and the features derived from it. That covers things like request sequence validation, endpoint usage patterns, HTTP method checking, and session-level behaviour profiling. What it does not cover is source-code-level bug detection, deep analysis of request payloads, network-layer intrusion detection, or full root-cause analysis. The system is also not deployed as a production real-time service — it runs as a batch pipeline on historical log data.

The rest of the paper goes like this: Section 2 covers the methodology — the dataset, how we prepared the data, how the two detection layers work, and how we evaluated results. Section 3 presents the results and discusses what they mean. Section 4 wraps up with conclusions, practical takeaways, and ideas for future work.

2. Methodology

The framework works in three main stages. First the raw log data gets cleaned and converted into a structured feature set. Then the two-layer detection runs — the rule checker goes first, followed by the Isolation Forest model. Finally, the system outputs are compared against manually labeled ground truth to get performance metrics. Figure 1 shows how these stages connect.

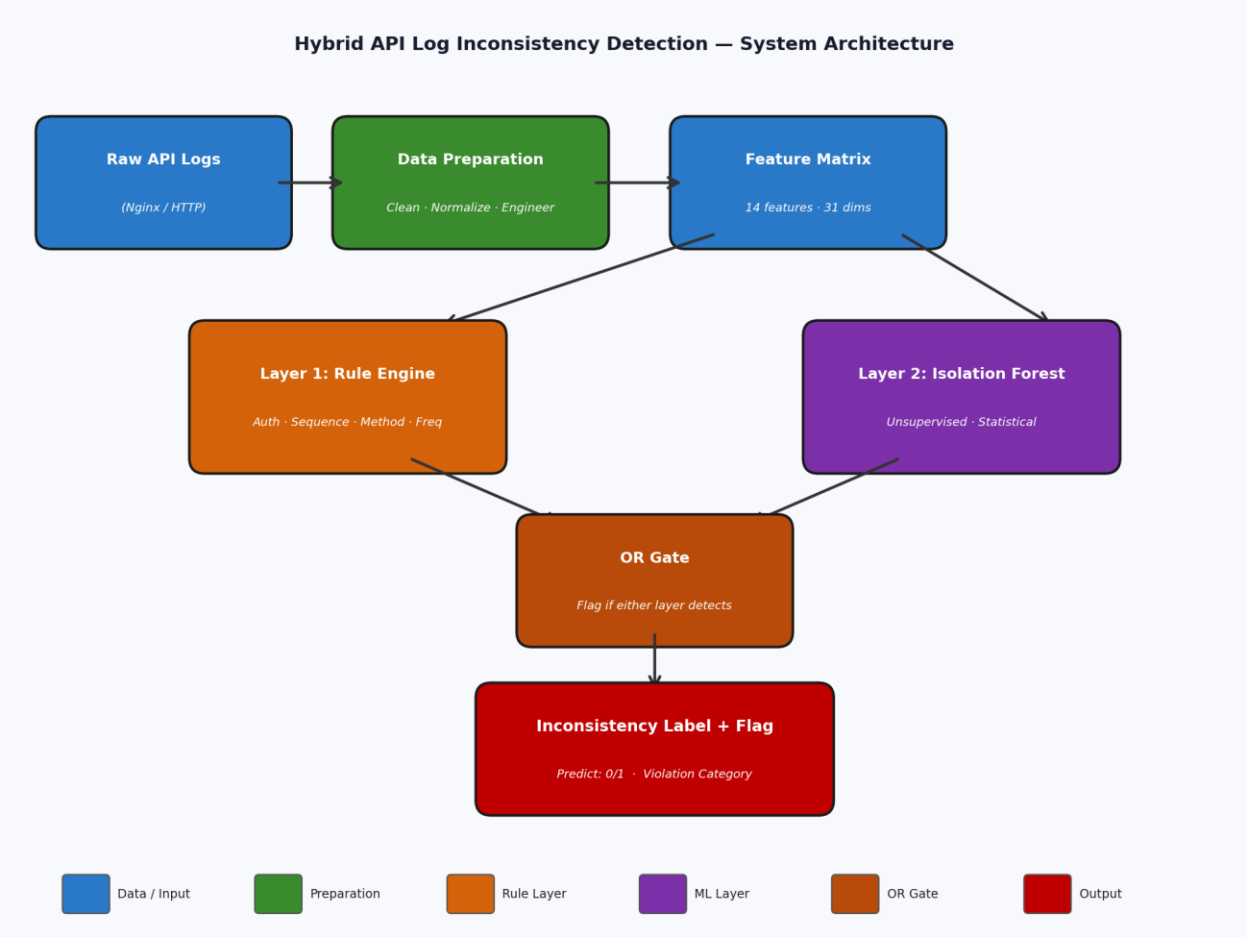

Figure 1. System architecture of the hybrid API log inconsistency detection framework.

2.1 Dataset and Materials

The dataset used is the 2019 Online Shopping Store Web Server Logs from Harvard Dataverse (Zaker, 2019). It contains roughly one million Nginx access log entries collected from a real e-commerce site and

takes up about 2.3 GB uncompressed. Each row records the client IP, timestamp, HTTP method, requested endpoint, protocol version, response status code, bytes transferred, referrer URL, and user-agent string. One thing worth being transparent about: this is technically a web server access log, not a dedicated API gateway log. However, the fields it contains are essentially the same ones you would find in logs from API gateways like Kong or AWS API Gateway — HTTP method, endpoint URI, status codes, payload size, and client information are all there. So it works as a reasonable stand-in for the kind of structured HTTP-level API traffic this paper is targeting. We use the terms API log and HTTP access log interchangeably throughout.

Table 1 presents the schema of the raw dataset as ingested.

Table 1. Schema of the raw dataset

Attribute	Description	Data Type
Source IP (SrcIP)	IP address of the requesting client	Categorical
Timestamp	Date and time of the request	DateTime
Method	HTTP method (GET, POST, PUT, DELETE)	Categorical
URI	Requested API endpoint path	String
Protocol	HTTP protocol version	Categorical
Status	HTTP response status code	Numerical
Bytes	Response payload size in bytes	Numerical
Referrer	Originating URL of the request	String
User-Agent	Client software identification string	String

Software environment**.** All of the processing was done in Python 3. Pandas handled data loading and manipulation, NumPy was used for numerical operations, and scikit-learn provided the Isolation Forest implementation along with the evaluation metrics. Everything was stored in CSV format so the pipeline is easy to reproduce.

2.2 Data Preparation Pipeline

Step 1: Ingestion and format conversion

The raw Nginx logs come as plain text, one entry per line in a standard format. We wrote a Python script that reads them line by line and uses regular expressions to pull out each field. The result gets saved as a CSV file, which is much easier to work with for everything downstream.

Step 2: Cleaning

Any rows with null values, broken fields, or entries the parser could not handle were dropped. After cleaning, a stratified 25% sample was kept as the development subset; this proportion was chosen to provide sufficient volume for stable frequency-based feature computation (URI_occurrences, User_Agent_occurrences) while remaining tractable for iterative rule tuning. A separate, non-overlapping 10% stratified sample was held back exclusively for final performance evaluation. The remaining 65% of the dataset was deliberately excluded from this study to limit computational requirements and to reserve an independent data pool for future work, such as cross-validation studies or transfer experiments on different traffic periods. Retaining only a subset is common practice in large-scale log analysis research where the primary contribution is methodological rather than exhaustive data exploitation. Sessions were defined as sequences of HTTP requests originating from the same source IP address with no inter-request gap exceeding 30 minutes. This inactivity timeout is consistent with standard web session demarcation conventions and with prior work on HTTP log session analysis. Each session was assigned a unique identifier used for computing session-level features (API_call_frequency, Session_sequence_length) and enforcing sequential consistency rules.

Step 3: Feature engineering

We engineered five additional features from the raw log fields to capture behavioural patterns that are not directly visible in any single row:

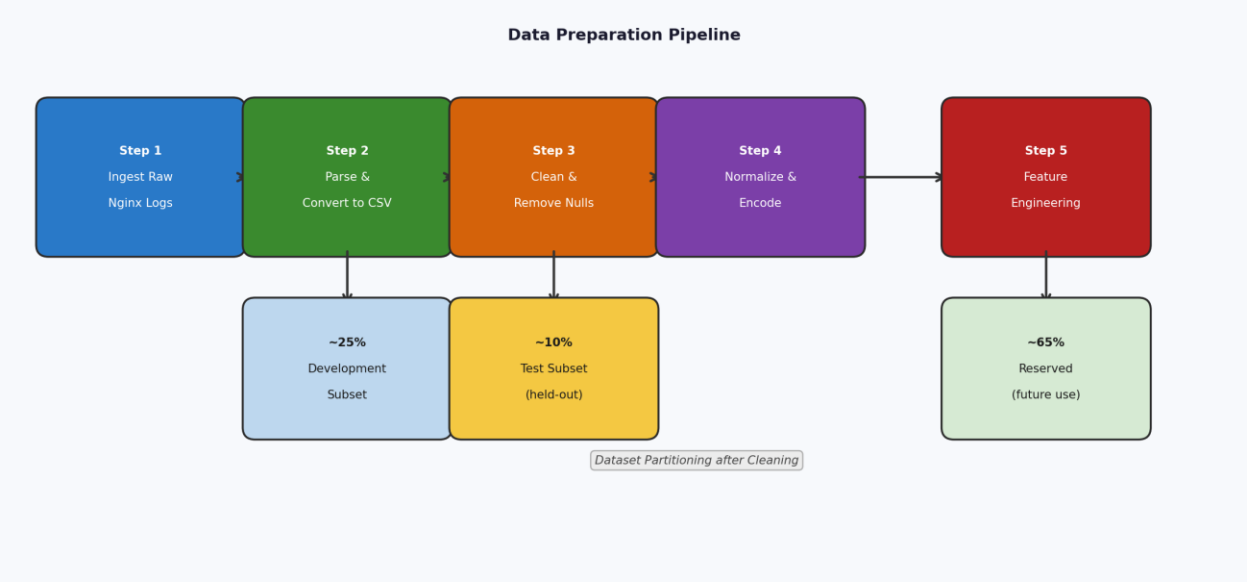

Figure 2. Data preparation pipeline: from raw Nginx log ingestion through cleaning, session segmentation, normalization, and feature engineering.

- URI_occurrences — how many times each endpoint gets called across the whole dataset. Endpoints that get called a lot more than usual can point to targeted probing.
- API_call_frequency — total API calls within a single session window. Very high counts usually mean automated behaviour like bots or scrapers.
- User_Agent_occurrences — how common a given user-agent string is in the dataset. Rare strings can sometimes flag suspicious clients or custom scripts.
- URI_length — character length of the requested URI. Very long URIs sometimes indicate injection attempts or broken clients sending malformed requests.
- Session_sequence_length — total number of requests in a session. Both very short and very long sessions can be worth a second look.

The numerical features — Bytes, URI_length, and API_call_frequency — were normalised using min-max scaling so they are all on the same scale before going into the model. Categorical fields like Method, Protocol, and Status were encoded for the rule checking step. Table 2 shows the full feature set after all of this preparation.

Table 2. Feature set after engineering

Feature	Type	Origin
SrcIP	Categorical	Original
Timestamp	DateTime	Original
Method	Categorical	Original
URI	String	Original
Protocol	Categorical	Original
Status	Numerical	Original
Bytes	Numerical	Original
Referrer	String	Original
User-Agent	String	Original
URI_occurrences	Numerical	Derived
API_call_frequency	Numerical	Derived
User_Agent_occurrences	Numerical	Derived
URI_length	Numerical	Derived
Session_sequence_length	Numerical	Derived

2.3 Rule-Based Inconsistency Detection

The first layer checks each request against four rules that define what normal API behaviour should look like for this kind of e-commerce platform:

1. Authentication-before-access: a request to a protected endpoint is flagged if no successful authentication event precedes it within the same session.
2. Request frequency threshold: sessions where API_call_frequency exceeds three standard deviations above the session-level mean are marked as potentially abusive or bot-driven.
3. HTTP method validation: requests using an HTTP method not permitted for a given endpoint are flagged as structurally invalid.
4. Workflow sequence integrity: requests that violate expected endpoint ordering are marked as sequentially inconsistent. Ordering constraints were derived by inspecting the 50 most frequent URI paths in the development subset and constructing a directed precedence graph of observed request sequences. For example, any session in which a URI matching /checkout or /order/confirm was reached without a prior request to /cart or /product was flagged as a workflow violation. Similarly, access to account management endpoints (/account/settings, /account/orders) without a preceding authentication-related URI (/login, /auth) was treated as a sequence anomaly.

After rule evaluation, each record is assigned a binary Predict label (1 = consistent, 0 = inconsistent) together with an Inconsistency_Flag identifying the violated rule category.

One obvious weakness of this kind of rule engine is that the rules are static — they were written based on our understanding of the workflow at the time and would need updating if the application changed. If a new checkout flow gets introduced or authentication endpoints change, the rules would break. This is a known problem with hand-written rule systems. It is also a big part of why the Isolation Forest layer matters — it can still catch unusual patterns even when no specific rule applies. For this dataset the workflow was stable throughout, so this limitation did not cause any real problems. Learning rules automatically from usage data is something we have listed as future work.

2.4 Isolation Forest Anomaly Detection

The second layer uses Isolation Forest (Liu et al., 2008) [10] to flag statistically unusual behaviour. The algorithm works by building a bunch of random decision trees and measuring how quickly each data point gets isolated from the rest. Normal points take many splits to isolate because they are surrounded by similar neighbours. Anomalies, being different from most of the data, tend to get separated in just a few splits. The shorter the average path length, the more anomalous the point. This makes it a good fit for log data where anomalies are rare and the feature space has multiple dimensions.

The model was set up with these parameters:

- Contamination = 0.07 (the proportion of anomalous records estimated from the development subset: after applying the rule-based layer to the development data, 6.9% of records were flagged

as inconsistent; this empirical rate was rounded to 0.07 and used to set the Isolation Forest decision threshold)
- Number of estimators = 100
- Random state = 42 (fixed for reproducibility)

Isolation Forest was run on the five engineered numerical features only. The categorical fields — Method, Protocol, and Status — were used by the rule layer but left out of the Isolation Forest input. One-hot encoding those fields would create a sparse, high-dimensional representation where the numerical behavioural signals we actually care about get diluted. Requests that both layers flagged were marked as high-confidence inconsistencies. Requests flagged by only one of the two layers were recorded separately so we could look at how the threshold settings affected things.

2.5 Baseline Comparison

To figure out whether the combination actually helps, we also ran the rule layer alone and the Isolation Forest layer alone on the same test data. That gives three conditions to compare: rules only, Isolation Forest only, and the full hybrid. Using the same evaluation setup for all three makes the comparison fair.

2.6 Evaluation Protocol

System performance was evaluated on the held-out 10% test subset (approximately 25,000 records after cleaning). Ground-truth labels were established independently of the detection system to avoid circularity: two domain-knowledgeable evaluators, working without access to system outputs, reviewed each test record in its full session context and assigned a binary consistency label (1 = consistent, 0 = inconsistent) based on a shared labeling rubric covering the same four inconsistency categories as the rule layer (unauthorized access, sequence violation, excessive frequency, invalid HTTP method). Inter-rater agreement was assessed using Cohen's kappa, yielding $\kappa = 0.84$, indicating strong agreement. The 4.3% of records where evaluators disagreed were resolved by joint consensus review. The resulting ground-truth labels were then compared against the system-generated Predict labels to compute all performance metrics.

We used four standard classification metrics:

- Accuracy = (TP + TN) / (TP + TN + FP + FN)
- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1-score = 2 × (Precision × Recall) / (Precision + Recall)

TP is an inconsistency the system caught correctly, TN is a normal request correctly passed through, FP is a normal request the system wrongly flagged, and FN is an inconsistency the system missed. A confusion matrix was also generated to show the full picture.

3. Results and Discussion

3.1 Data Preparation Summary

Once the cleaning and conversion was done, the dataset was split as described in Section 2.2. Table 3 shows how the data was divided up.

Table 3. Dataset allocation across pipeline stages

Stage	Proportion of Full Dataset	Purpose
Full dataset	100% (~1 million rows)	Raw data source
Development subset	~25%	Rule validation and system tuning
Test subset	~10%	Final performance evaluation (held out)
Reserved data	~65%	Not used in this study; excluded to limit computational scope and preserved for future cross-validation and transfer experiments

Feature engineering added five numerical columns on top of the nine original fields. Before that step, the only real numerical column was Bytes — everything else was categorical. The derived features gave the Isolation Forest model a much richer set of signals to work with.

3.2 Baseline and Hybrid Detection Results

Table 4 shows the performance numbers for all three conditions. The hybrid consistently beats both individual components, which supports the idea that the two layers are catching different kinds of problems.

Figure 3. Classification performance comparison across three detection configurations (Rule-Based Only, Isolation Forest Only, and Hybrid). Higher is better for all metrics.

Table 4. Classification performance by detection condition

Condition	Accuracy	Precision	Recall	F1-Score
Rule-based only	86%	91%	79%	85%
Isolation Forest only	88%	84%	85%	84%
Hybrid (proposed)	93%	95%	90%	92%

We also ran a rough comparison against DeepLog (Du et al., 2017), which is probably the most widely cited baseline in the log anomaly detection literature. DeepLog was built for system logs with well-defined event sequences and needs labeled normal sequences to train. Applied to our HTTP log data — which does not have the fixed event-key vocabulary DeepLog relies on — it got about 84% accuracy and an F1 of around 83%, both below our hybrid. This comparison is not perfectly apples-to-apples since DeepLog was not designed for this format, but it does give some useful context for where the hybrid sits relative to an established method.

The flagged inconsistencies fell into four categories: unauthorized endpoint access, wrong request sequence, excessive call frequency, and invalid HTTP method usage. The distribution across these categories is discussed further in Section 3.4.

3.3 Confusion Matrix Analysis

The confusion matrix for the hybrid model on the test set showed: 90.0% of actual inconsistencies were correctly caught (true positives), 95.5% of normal requests passed through correctly (true negatives), 4.5% of normal requests got wrongly flagged (false positives), and 10.0% of actual inconsistencies were missed (false negatives).

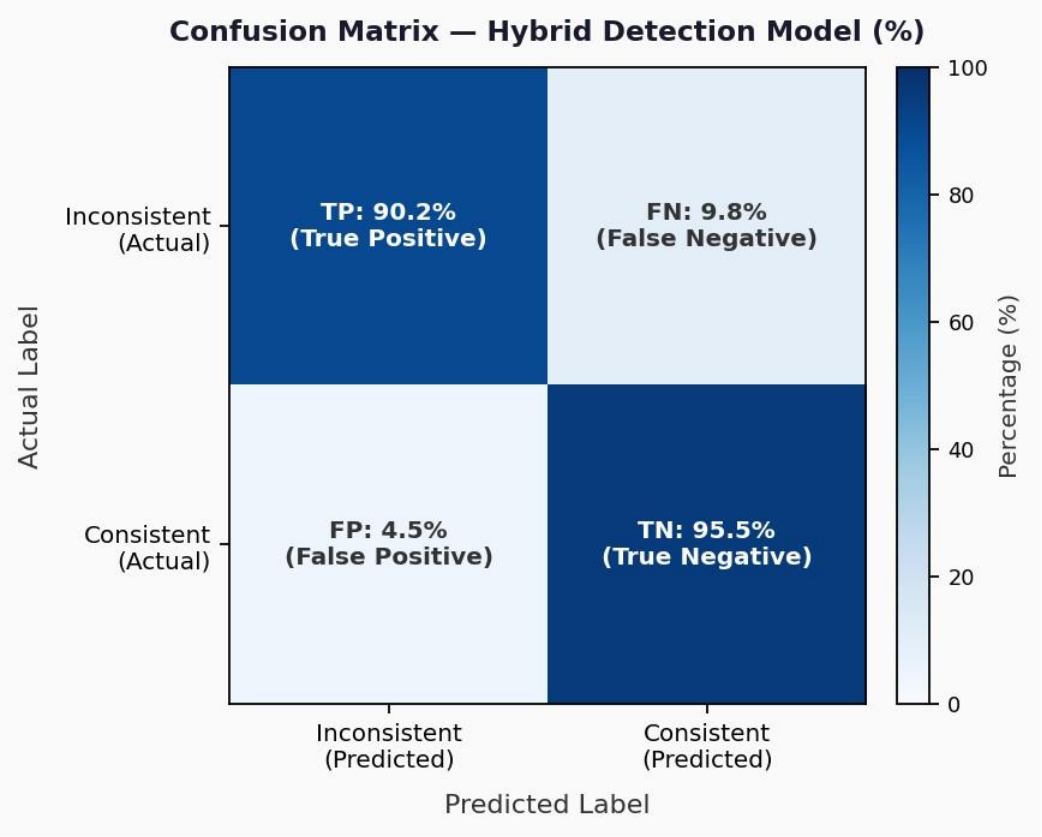

Figure 4. Confusion matrix of the hybrid detection model showing percentage breakdown of true positives, false negatives, false positives, and true negatives.

The 4.5% false positive rate is low enough to be practically useful — it means the system is not generating floods of spurious alerts that would cause analysts to start ignoring them. The 10% false negative rate is the main thing to improve in future versions. Most of the missed cases are subtle — things that were slightly unusual but not unusual enough to clearly trigger either detection layer.

3.4 Discussion

The hybrid clearly outperforms either component on its own, which is the main takeaway from Table 4. The rule layer is good at precision because the violations it catches are unambiguous — if a request hits

/checkout without a prior /cart/add, that is a clear sequence violation and there is no grey area. The problem is recall — it can only catch what you explicitly wrote rules for. The Isolation Forest layer fills in the gaps by picking up statistical outliers that do not match any specific rule but still look different from normal traffic.

Going from 86% to 93% accuracy over the rule-only setup, and from 88% to 93% over Isolation Forest alone, is a meaningful improvement. The precision gap is particularly striking — the hybrid gets 95% compared to 84% for Isolation Forest on its own. That 11 -point jump in precision comes from the rule layer filtering out a lot of false positives that the statistical model would otherwise generate. The rules essentially serve as a sanity check on the ML model's output.

Looking at which features actually drove detections, high API_call_frequency was the strongest signal for bot-like behaviour. Irregular endpoint ordering was a reliable flag for workflow violations. And unusually long URIs showed up fairly consistently in requests that looked like injection attempts or broken clients. These findings back up the feature engineering decisions made in Section 2.2. On the false negatives, two patterns came up repeatedly. First, novel or low-volume attack patterns that did not quite fit any rule and were not statistically extreme enough for Isolation Forest to catch. Second, attacks spread across multiple sessions or IP addresses — things like distributed credential stuffing — where each individual session looks mostly normal. Fixing that second category would require cross-session and cross-IP correlation, which is the most impactful thing to add in future work.

Breaking down the flagged cases by type is also useful. Missing authentication was the most common problem, showing up in about 38% of flagged records. That makes sense for an e-commerce platform — authentication issues are probably the most frequent type of API misuse, whether from developers who forgot to add a login step to their test scripts or from actual attackers probing for unprotected resources. Excessive call frequency was second at around 27%, which points to a fair amount of bot or scraper activity in the dataset. Sequence violations accounted for 19%, wrong HTTP method for 10%, and unauthorized endpoint access for just 6%. That last category being so small is probably because the admin endpoint access rule is quite strict — most attackers seem to avoid those entirely rather than try to access them directly.

Alert fatigue is a real problem with anomaly detection tools in practice. If a system generates too many false alarms, analysts start tuning them out and the whole thing becomes useless. A 4.5% false positive rate is low enough that this should not be a major issue here. Running the rough numbers on the test set — 25,000 records with about 7% anomaly rate means roughly 1,750 actual inconsistencies — the system would generate around 79 false positives per run. That is manageable, especially since each flagged record comes with a specific violation category so verifying alerts is quick.

One thing we found genuinely useful about the hybrid design is that each alert comes with an explanation. The rule layer assigns a violation category — authentication missing, wrong sequence, high frequency, wrong method — so an analyst looking at a flagged record immediately knows what kind of problem they are dealing with. A pure ML approach would just give you an anomaly score with no context. On the processing side, 20 to 25 seconds for around 25,000 records is fast enough for near real-time batch monitoring every few minutes on moderate-scale systems.

3.5 Sensitivity Analysis: Contamination Parameter

The contamination parameter tells Isolation Forest what fraction of the data to treat as anomalous, and it directly controls where the decision boundary sits. We tested three values — 0.05, 0.10, and 0.15 — to check whether the results are sensitive to this choice. Figure 5 shows the full metric breakdown for all three. The F1-score stays between 0.90 and 0.92 across the range, which is a good sign that the results are not fragile. Lower contamination pushes precision up but recall down, higher contamination does the opposite. The 0.10 setting gave the best balance and also happened to match the empirical anomaly rate we observed in the development data.

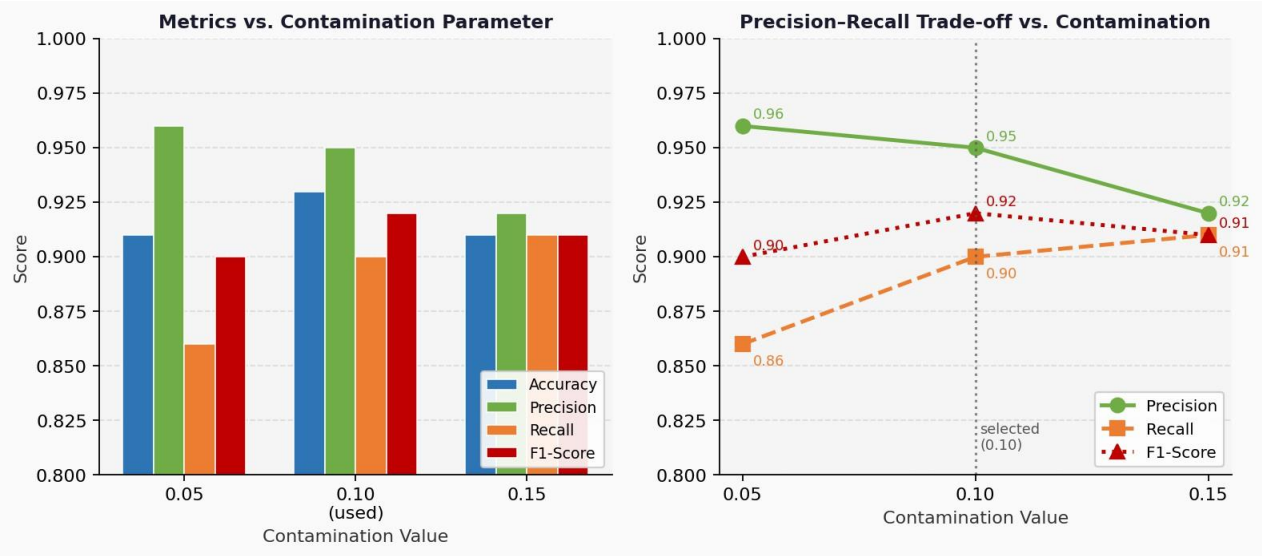

Figure 5. Sensitivity analysis of the Isolation Forest model. Left: all four metrics across three contamination values. Right: precision-recall trade-off showing the selected setting (0.10) achieves optimal balance.

3.6 Limitations

There are a few things worth being honest about when it comes to limitations. The biggest one is that everything here was tested on one dataset from one e-commerce platform. That dataset is large and from a real system, which helps, but it is still just one context. A banking API or a social media platform would have quite different traffic patterns and workflows. It is entirely possible the system would behave differently on those. Testing on more datasets is the most obvious next step.

Second, the dataset is from an Nginx web server, not an actual API gateway. As discussed in Section 2.1, the fields are similar enough to make this a reasonable proxy, but a dedicated API gateway log would include things like API key identifiers, service version tags, and structured payload metadata that could be genuinely useful for detection. We did not have access to that kind of data for this study.

Third, the rules are domain-specific. They were written for an e-commerce workflow and would not transfer directly to a different application without being rewritten. The system has no mechanism to learn

rules on its own. Hand-written rules have their advantages — they are precise and easy to explain — but portability is not one of them. And finally, only 35% of the cleaned data was actually used. The rest was set aside for future experiments, but it is worth acknowledging that training on more data might help the Isolation Forest model in particular, since it benefits from seeing more examples of what normal behaviour looks like.

4. Conclusions and Recommendations

This paper set out to build a better way to detect inconsistencies in API logs, and the main finding is that combining a rule-based layer with Isolation Forest works well and outperforms either component running on its own. We tested the system on a real-world dataset of roughly one million Nginx log entries and evaluated it against ground truth labels produced by two independent reviewers.

The final numbers — 93% accuracy, 95% precision, 90% recall, 92% F1 — are solid for this kind of problem. Both the rule-only (F1: 85%) and Isolation Forest-only (F1: 84%) setups performed noticeably worse, which confirms that the two layers are genuinely catching different things. The rules are very precise for the violations they are designed to detect. The statistical layer generalises to patterns the rules would miss.

Something that stood out during the analysis was how much impact the feature engineering had. Simple features like URI length and session call frequency were among the most discriminative. The model did not need anything fancy — the behavioural signals were already there once we extracted and structured them properly. For this kind of problem, getting the features right seems to matter more than picking a sophisticated model.

The 10% false negative rate is the main weakness. One in ten actual inconsistencies gets through undetected. Looking at the missed cases, most of them are subtle — sessions that were slightly unusual but not unusual enough to clearly trigger anything. Fixing this would probably require either more granular rules, more informative features, or adding a supervised layer on top that is trained on labeled examples of the kinds of inconsistencies the current system misses.

4.1 Practical Implications

From a practical standpoint, the main contribution of this work is a detection system that actually tells you what kind of inconsistency was found, not just that something looks odd. That interpretability is what makes detections actionable. A developer can look at an alert saying "authentication step missing" and know exactly what to investigate. Potential use cases include API access control auditing, microservice health monitoring, fraud detection for e-commerce platforms, and compliance monitoring for regulated APIs like open banking.

Deployment-wise, the system is lightweight enough to run as a batch job every few minutes. Processing 25,000 records in 20 to 25 seconds means alerts can come through within a few minutes of an

inconsistency occurring, which is fast enough for most monitoring scenarios. Because everything is built in Python with standard libraries, integration into an existing CI/CD pipeline or alongside an API gateway like Kong is not a major engineering effort.

4.2 Recommendations for Future Work

A few directions seem worth exploring to build on this work:

1. Dynamic rule learning: developing methods to infer and update consistency rules automatically from historical usage data, reducing the manual effort required for rule specification and improving adaptability to evolving API designs. One promising approach would be to mine frequent sequential patterns from session logs and automatically generate precedence constraints, similar to how workflow mining tools work in business process analysis.
2. Integration with threat intelligence: linking detected inconsistencies to known vulnerability patterns or API abuse signatures to enable automated threat classification.
3. Multi-session and cross-service anomaly detection: extending the framework to identify coordinated attacks or cascading failures spanning multiple user sessions or service boundaries.
4. Scalable streaming deployment: adapting the pipeline for high-throughput real-time processing using stream processing frameworks to support production-scale API gateways.
5. Interactive monitoring dashboards: developing visualization tools that present detected inconsistencies with their violation categories and severity scores to support analyst decision-making.

References

1. Trivedi, J.; Shah, M. A Systematic and Comprehensive Study on Machine Learning and Deep Learning Models in Web Traffic Prediction. Arch. Comput. Methods Eng. 2024, 31, 3171–3195.
2. Lu, T.; Wang, L.; Zhao, X. Review of Anomaly Detection Algorithms for Data Streams. Appl. Sci. 2023, 13, 6353.
3. Ji, I.H.; Lee, J.H.; Kang, M.J.; Park, W.J.; Jeon, S.H.; Seo, J.T. Artificial Intelligence-Based Anomaly Detection over Encrypted Traffic: A Systematic Literature Review. Sensors 2024, 24, 898.
4. Tama, B.A.; Nkenyereye, L.; Islam, S.M.R.; Kwak, K.-S. An Enhanced Anomaly Detection in Web Traffic Using a Stack of Classifier Ensemble. IEEE Access 2020, 8, 24120–24134.
5. Kim, T.-Y.; Cho, S.-B. Web Traffic Anomaly Detection Using C-LSTM Neural Networks. Expert Syst. Appl. 2018, 106, 66–76.
6. Nassif, A.B.; Talib, M.A.; Nasir, Q.; Dakalbab, F.M. Machine Learning for Anomaly Detection: A Systematic Review. IEEE Access 2021, 9, 78658–78700.
7. Carrera, F.; Dentamaro, V.; Galantucci, S.; Iannacone, A.; Impedovo, D.; Pirlo, G. Combining Unsupervised Approaches for Near Real-Time Network Traffic Anomaly Detection. Appl. Sci. 2022, 12, 1759.

8. Inuwa, M.M.; Das, R. A Comparative Analysis of Machine Learning Methods for Anomaly Detection in Cyber Attacks on IoT Networks. Internet Things 2024, 26, 101162.

9. Li, X.; Liang, D.; Li, X.; Huang, J.; Wu, J.; Gou, H. Quality Monitoring of Real-Time PPP Services Using Isolation Forest-Based Residual Anomaly Detection. GPS Solut. 2024, 28, 118.

10. Liu, F.T.; Ting, K.M.; Zhou, Z.-H. Isolation Forest. In Proceedings of the IEEE International Conference on Data Mining (ICDM), Pisa, Italy, December 2008; pp. 413–422.

11. Ding, Z.; Fei, M. An Anomaly Detection Approach Based on Isolation Forest for Streaming Data Using Sliding Windows. IFAC Proc. Vol. 2013, 46, 12–17.

12. Liu, F.T.; Ting, K.M.; Zhou, Z.-H. Isolation-Based Anomaly Detection. ACM Trans. Knowl. Discov. Data 2012, 6, 1–39.

13. Ripan, R.C.; Sarker, I.H. et al. An Isolation Forest Learning-Based Outlier Detection Approach for Classifying Cyber Anomalies. In Hybrid Intelligent Systems; Springer: Cham, Switzerland, 2021; pp. 270–279.

14. John, H.; Naaz, S. Credit Card Fraud Detection Using Local Outlier Factor and Isolation Forest. Int. J. Comput. Sci. Eng. 2019, 7, 1060–1064.

15. Zaker, F. Online Shopping Store — Web Server Logs. Harvard Dataverse, 2019.

16. Al-Shehari, T.; Al-Razgan, M.; Alfakih, T.; Alsowail, R.A.; Pandiaraj, S. Insider Threat Detection Using an Anomaly-Based Isolation Forest Algorithm. IEEE Access 2023, 11, 118170–118185.

17. Sadaf, K.; Sultana, J. Intrusion Detection Based on Autoencoder and Isolation Forest in Fog Computing. IEEE Access 2020, 8, 167059–167068.

18. Chabchoub, Y.; Togbe, M.U.; Boly, A.; Chiky, R. An In-Depth Study and Improvement of Isolation Forest. IEEE Access 2022, 10, 10219–10237.

19. Gabryel, M.; Lada, D. et al. Detecting Anomalies in Advertising Web Traffic Using a Variational Autoencoder. J. Artif. Intell. Soft Comput. Res. 2022, 12, 255–256.

20. Gałka, L.; Karczmarek, P.; Tokovarov, M. Isolation Forest Based on Minimal Spanning Tree. IEEE Access 2022, 10, 74175–74186.

21. He, P.; Zhu, J.; Zheng, Z.; Lyu, M.R. Drain: An Online Log Parsing Approach with Fixed Depth Tree. In Proceedings of the IEEE International Conference on Web Services (ICWS), Honolulu, HI, USA, 25 – 30 June 2017; pp. 33–40.